

Tachometer Circuit

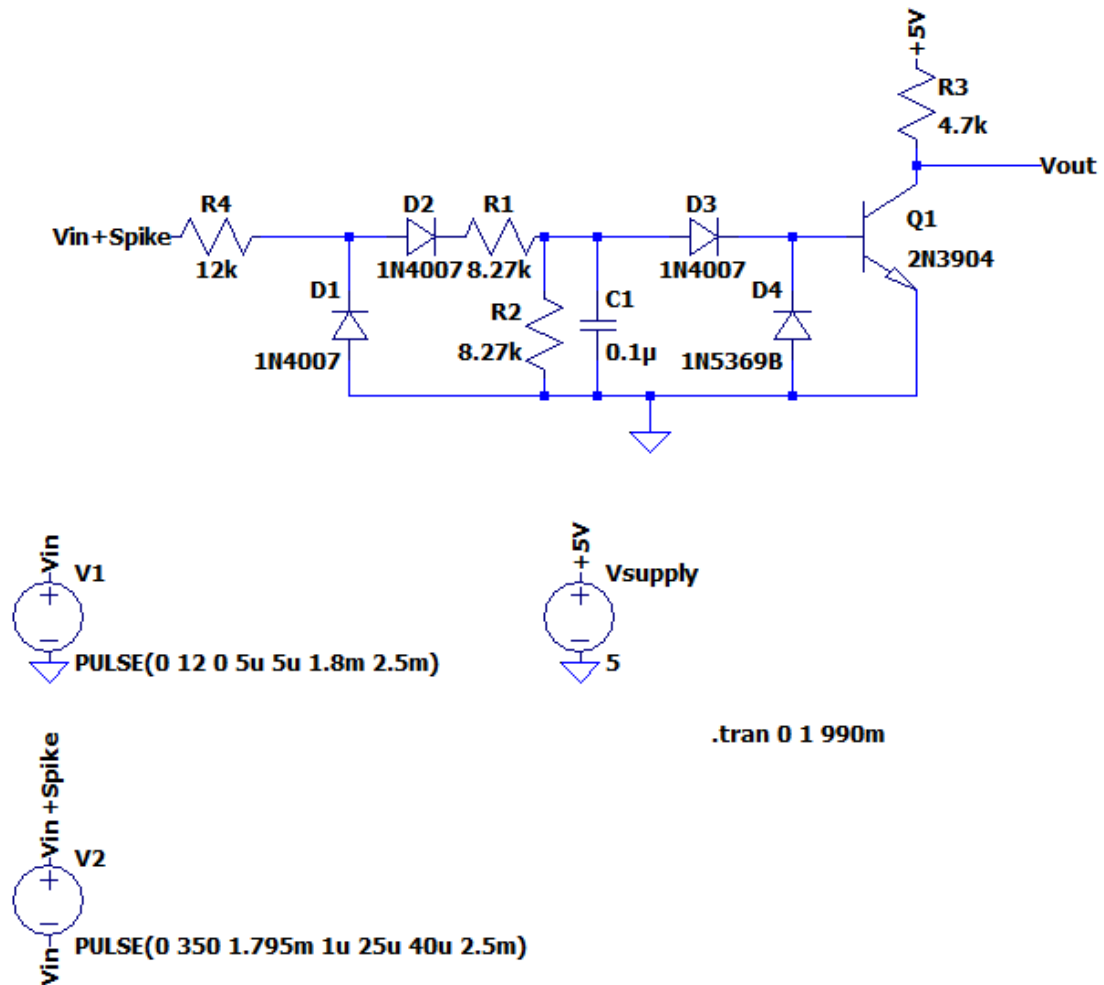
Jonathan Meeker

1. Purpose

- a. I need a tachometer for my 1967 mustang as it doesn't come with one. The price for buying one off the market is ridiculously high, so I figured I'd just make it

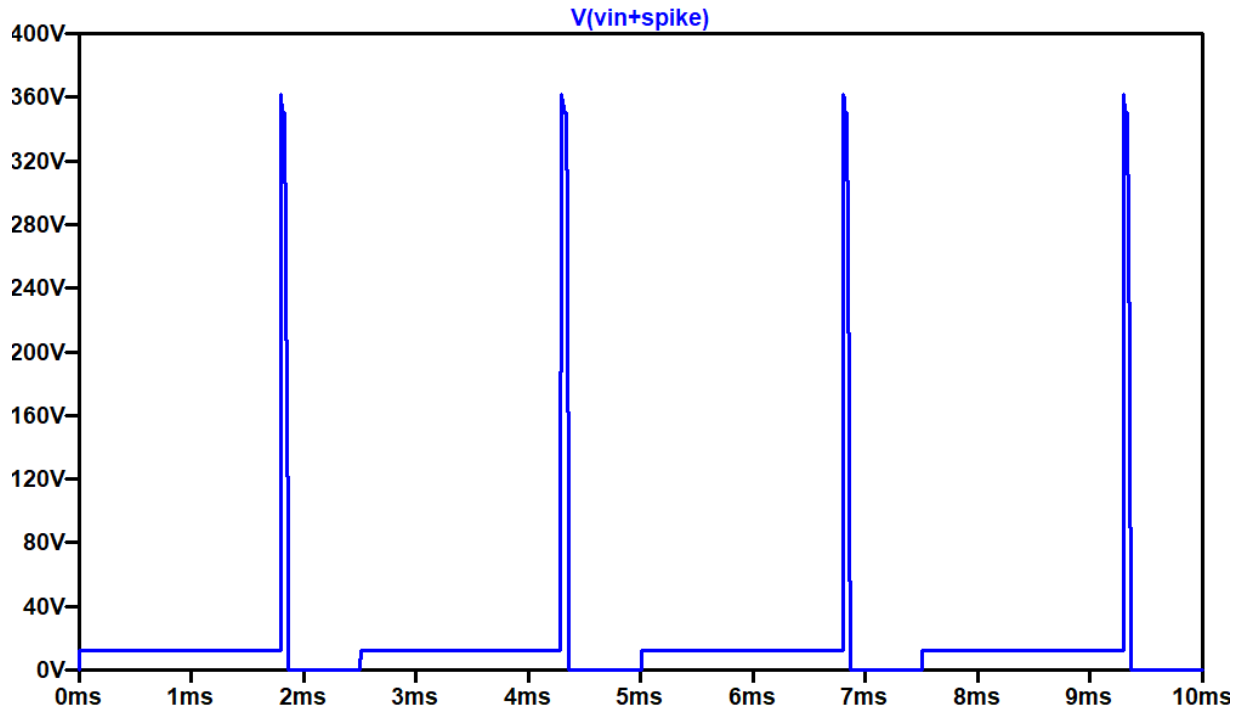
2. Design

- a. Overview: I can electronically read the rpm of my car by reading when the negative terminal of the coil activates each spark plug. I then want to output this reading using a needle gauge that I can mount on my dash
- b. Circuit design: The main issue with reading whenever the coil fires is that it needs to generate a high voltage to make the spark plug fire. The coil stays around 12V in idle and then drops to 0V when it fires. On the falling edge of that square wave, the spark ignites increasing the voltage to around 300-400V. This could damage any signal reading device I use to measure the frequency, so to avoid this I must design a circuit to handle the high voltage

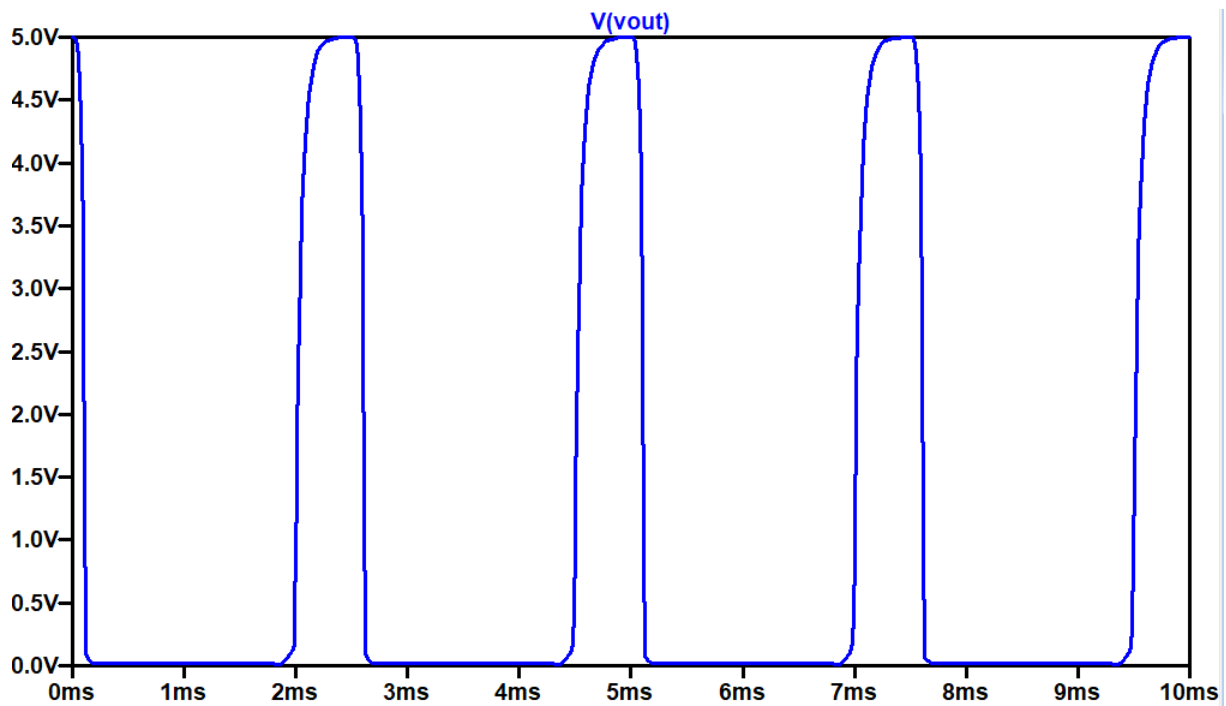


This circuit clamps the input voltage to around 6.1 Volts, which is pretty safe for an Arduino to read, while also filtering out EMI noise from the engine bay and handling the large voltage spike. The $R4$ input resistor helps reduce the current input to the circuit. The $D1$ diode prevents negative voltage from entering the transistor base or the Arduino. $R1$ and $R2$ voltage divide to further reduce incoming current and voltage. And the clamper circuit ensures no over voltage can damage any of the components. Finally, there is a pull up resistor to 5V to ensure no ambiguity for the Arduino to read when it fires. I'm simulating the coil spark and frequency with a 12V to 0 V square wave and a 350V spike on the falling edge of the circuit (which is when the spark plug fires)

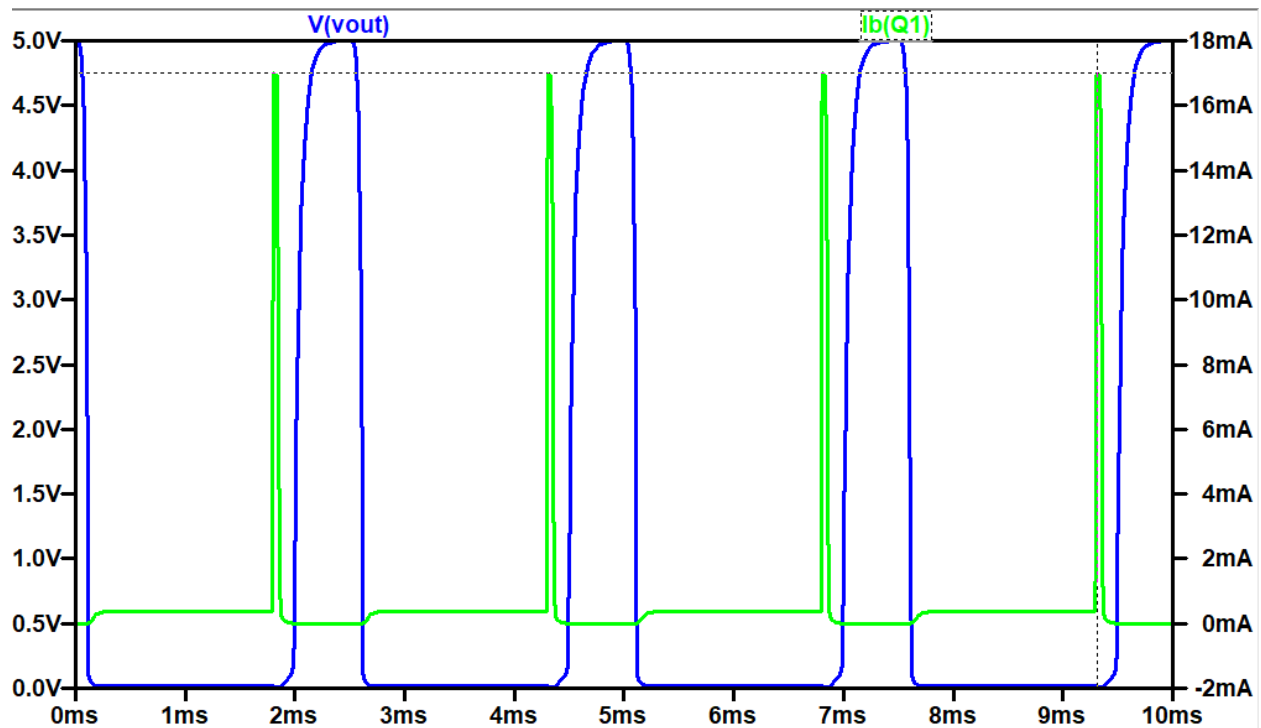
Simulated coil input for the circuit:



Simulated output of the circuit to the Arduino:



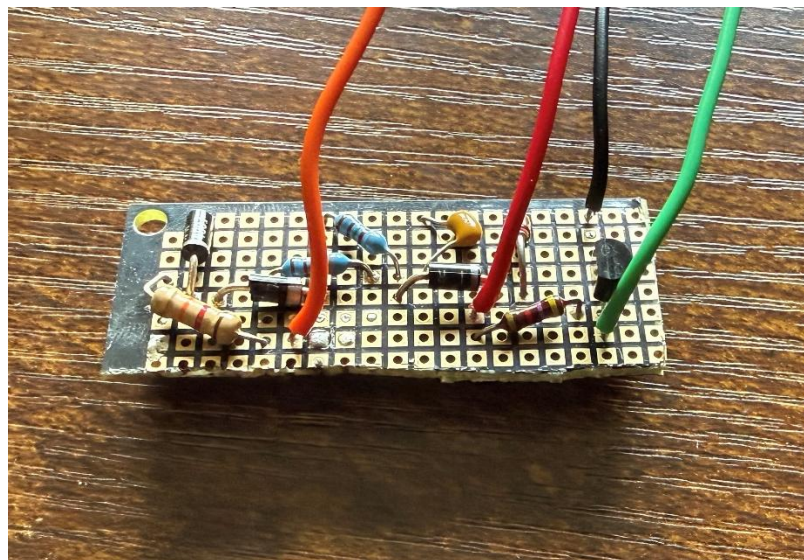
Simulated output and max current into the base of the transistor:



As expected, the circuit outputs a safe 5V to the Arduino and handles current appropriately (with a max current output of 17mA, well under the max peak current of around 100mA for the 2n3904)

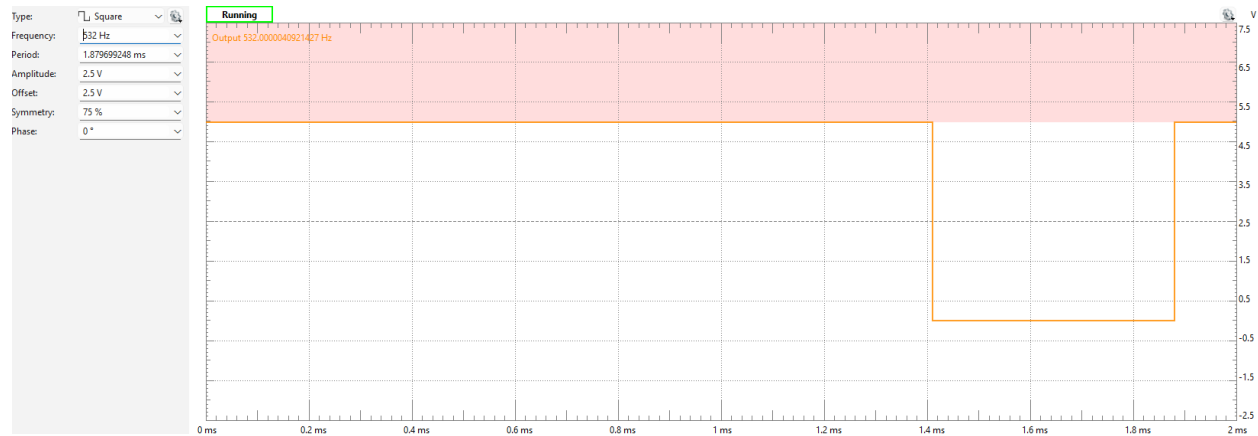
3. Progress

a. Built circuit

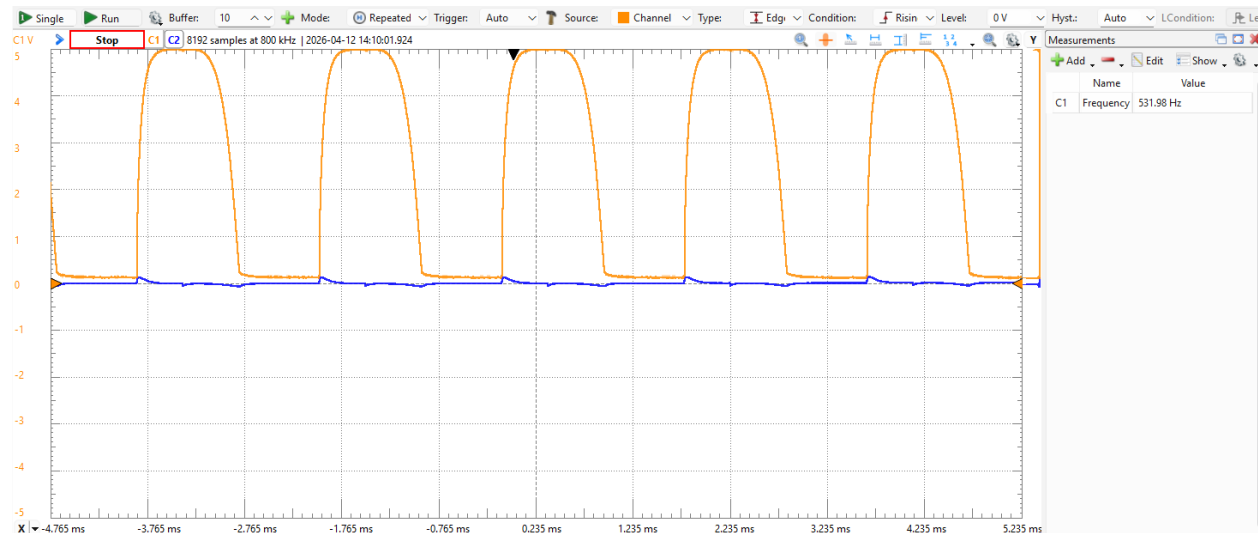


Starting with what works, This circuit successfully steps down a high input voltage and filters out noise. There are no concerns with the high spark voltage effecting the Arduino or NPN transistor.

Input wave shape:



Output for a 532Hz 5V to 0V square wave



The output looks relatively clean and will be able to interpret the frequency very easily on the Arduino. Note that 532Hz is equivalent to a max 8k RPM. The coil sparks 4 times every rotation of the engine and I am reading off the sparks of the coil, so the math is $(8000\text{rpm} \times 4\text{sparks/rotation}) / 60\text{s}$. This is the highest that I plan to output on this device so it is satisfactory to test up to this frequency.

Now for the mistakes. I made a couple errors on this part, but the biggest one is I didn't test the circuit on a breadboard first.

- i. Mistake #1: Not testing

1. Error: jumping to soldering. Fix: test first
 - a. I kept iterating repeatedly because I realized the built circuit was not exactly like the simulated circuit. I had to redo the current calculations to get it to a safe current, and then I had issues with the RC delay attenuating the transistor output, and then I realized that all of it was fine because frequency is measured in seconds, not minutes, so the high frequency that was causing the signal to attenuate is nowhere near the max the system would have to output. In summary, next time test on breadboard first, make all your mistakes and fix them, and then solder.
- ii. Mistake #2: Messy soldering
 1. Error: sloppy work. Fix: Do better
 - a. Not a lot to say here. I had to keep reiterating on my design which required me to desolder and resolder a bunch of times and left a lot of mess on the board. In the future I will account for all the variables before soldering

Arduino Code:

```
// =====  
// 1967 Mustang V8 Tachometer - 100ms Update Version  
// Fast & responsive - updates 10 times per second  
// =====  
  
const int tachPin = 2;  
  
volatile unsigned long pulseCount = 0;  
unsigned long lastTime = 0;  
unsigned long rpm = 0;  
  
const int cylinders = 8;          // V8 engine  
  
void setup() {  
  Serial.begin(115200);  
  pinMode(tachPin, INPUT_PULLUP);  
  
  attachInterrupt(digitalPinToInterrupt(tachPin), tachISR, FALLING);  
  
  Serial.println("Tachometer Ready - Updating every 100 ms (10x per second)");  
}
```

```

    Serial.println("Connect your AD2 and test different frequencies");
}

void loop() {
    if (millis() - lastTime >= 100) {           // Update every 100 ms
        noInterrupts();
        unsigned long count = pulseCount;
        pulseCount = 0;
        interrupts();

        // RPM calculation for V8 4-stroke (100ms = 1/10 second)
        // Multiplier = 120 * 10 = 1200
        rpm = (count * 1200UL) / cylinders;

        // Output to Serial Monitor
        Serial.print("Pulses in 100ms: ");
        Serial.print(count);
        Serial.print(" | RPM: ");
        Serial.println(rpm);

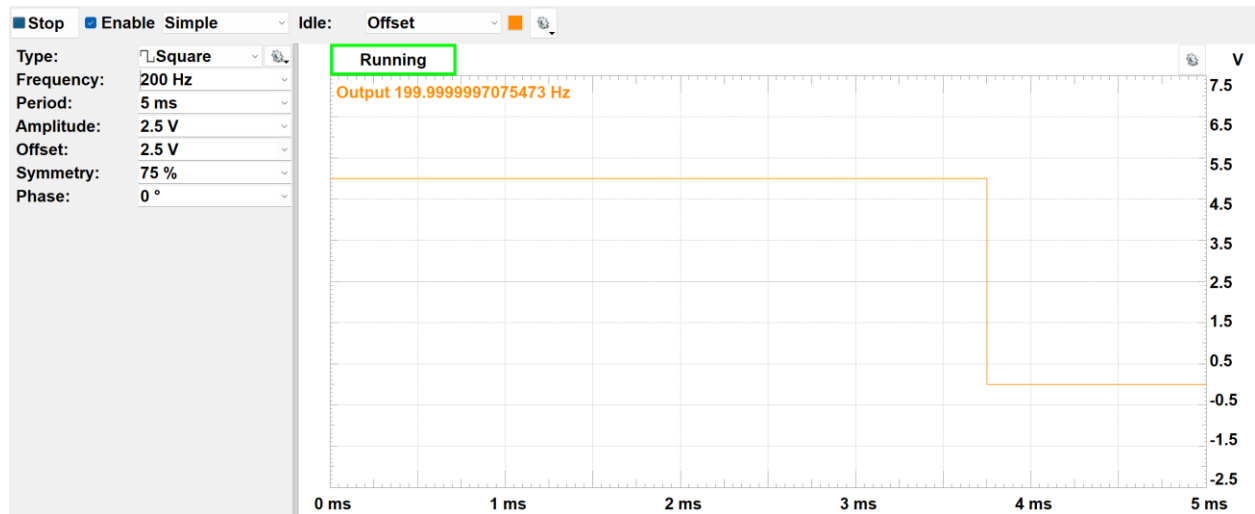
        lastTime = millis();
    }
}

// Interrupt Service Routine
void tachISR() {
    pulseCount++;
}

```

This code reads the frequency from the circuit and outputs the rpm value. It was generated using Grok.

Input from AD2:

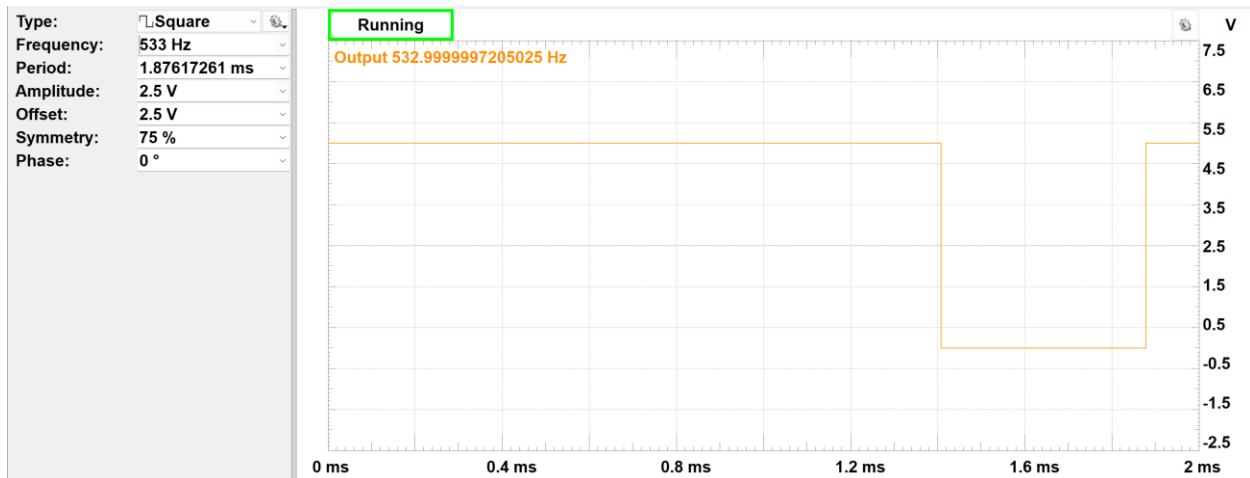


Output from Arduino:

```
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 200 | RPM: 3000
Pulses/sec: 199 | RPM: 2985
Pulses/sec: 199 | RPM: 2985
```

It is reading the frequency as expected. Now to check max frequency

Input frequency:



Output from Arduino:

```
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 530 | RPM: 7950
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 530 | RPM: 7950
Pulses/sec: 531 | RPM: 7965
Pulses/sec: 530 | RPM: 7950
```

It reads the max with high accuracy $(533-530.5)/533 = .4\%$ error. This is satisfactory as a benchmark.

For the servo motor circuit, the wiring was fairly straight forward. I connected the 4 legs on the back of the motor to the D4-D7 pins on the Arduino. I made sure to update the Arduino every 100ms to have a smooth needle output and limit the amount of interference from noise in the engine. The final code for the Arduino is shown below

```
#include <SwitecX25.h>

// ===== CONFIGURATION =====
```

```

const int tachPin = 2;                // Input from conditioner circuit
const int cylinders = 8;

const int maxRPM = 9000;
const int maxSteps = 820;            // ← Adjust to match your sticker

const int ledPin = 9;                // PWM pin for LED
const int ledBrightness = 180;       // ← Adjust brightness: 0-255 (180 is
good starting point)

SwitecX25 motor1(945, 4, 5, 6, 7);

// ===== VARIABLES =====
volatile unsigned long pulseCount = 0;
unsigned long lastTime = 0;
unsigned long currentRPM = 0;
unsigned long sparksPerSecond = 0;

void setup() {
  Serial.begin(115200);
  Serial.println("1967 Mustang Tachometer + Always-On LED");

  pinMode(tachPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);

  // Turn LED on at constant brightness
  analogWrite(ledPin, ledBrightness);

  attachInterrupt(digitalPinToInterrupt(tachPin), tachISR, FALLING);

  motor1.zero();
  delay(2000);

  Serial.println("Ready! LED is always on at constant brightness");
}

void loop() {
  // === RPM Calculation every 100ms ===
  if (millis() - lastTime >= 100) {
    noInterrupts();
    unsigned long count = pulseCount;
    pulseCount = 0;
    interrupts();

    sparksPerSecond = count * 10;
  }
}

```

```

currentRPM = (count * 1200UL) / cylinders;

// Map RPM to needle
int targetSteps = map(currentRPM, 0, maxRPM, 0, maxSteps);
targetSteps = constrain(targetSteps, 0, maxSteps);

motor1.setPosition(targetSteps);

// Serial Monitor output
Serial.print("Sparks/sec: ");
Serial.print(sparksPerSecond);
Serial.print(" | RPM: ");
Serial.print(currentRPM);
Serial.print(" | Steps: ");
Serial.println(targetSteps);

lastTime = millis();
}

// Keep motor smooth
motor1.update();
}

// Interrupt
void tachISR() {
  pulseCount++;
}

```

This code was generated using Grok. I adjusted the mapping to the sticker I end up putting on the gauge so that the max is where the 9k rpm is on the sticker and min is 0 on the sticker. Additionally I added a PWM function for an LED to make sure I can read the gauge at night.

The wiring diagram for the whole circuit is shown below:

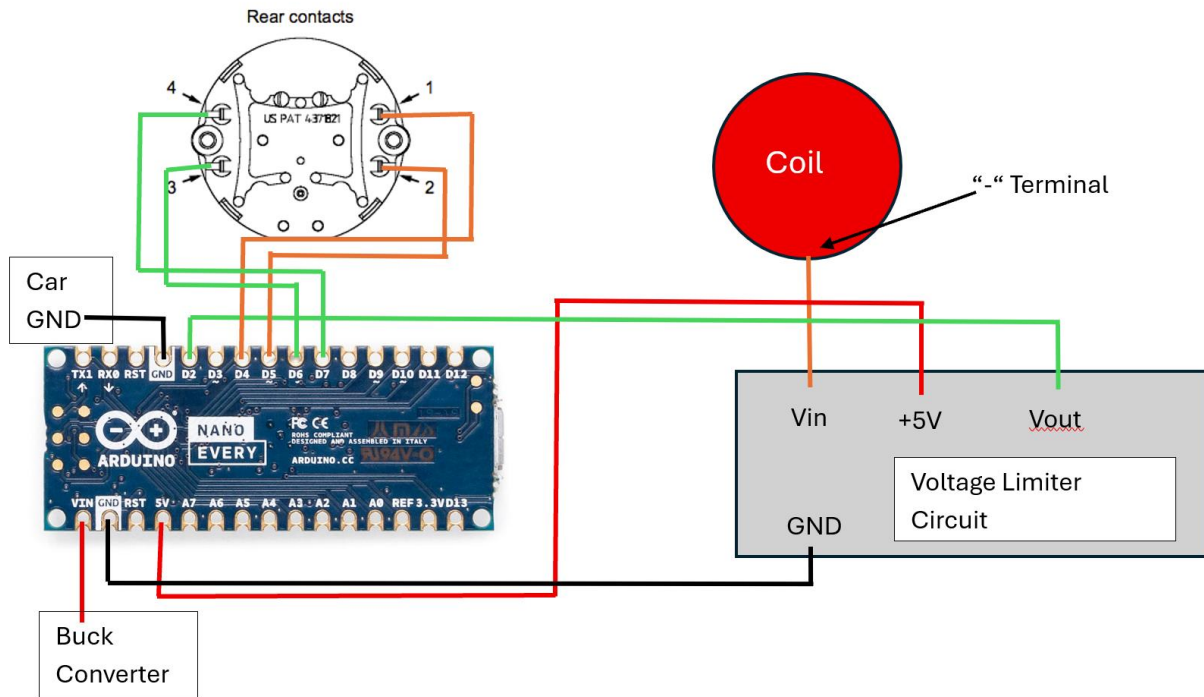
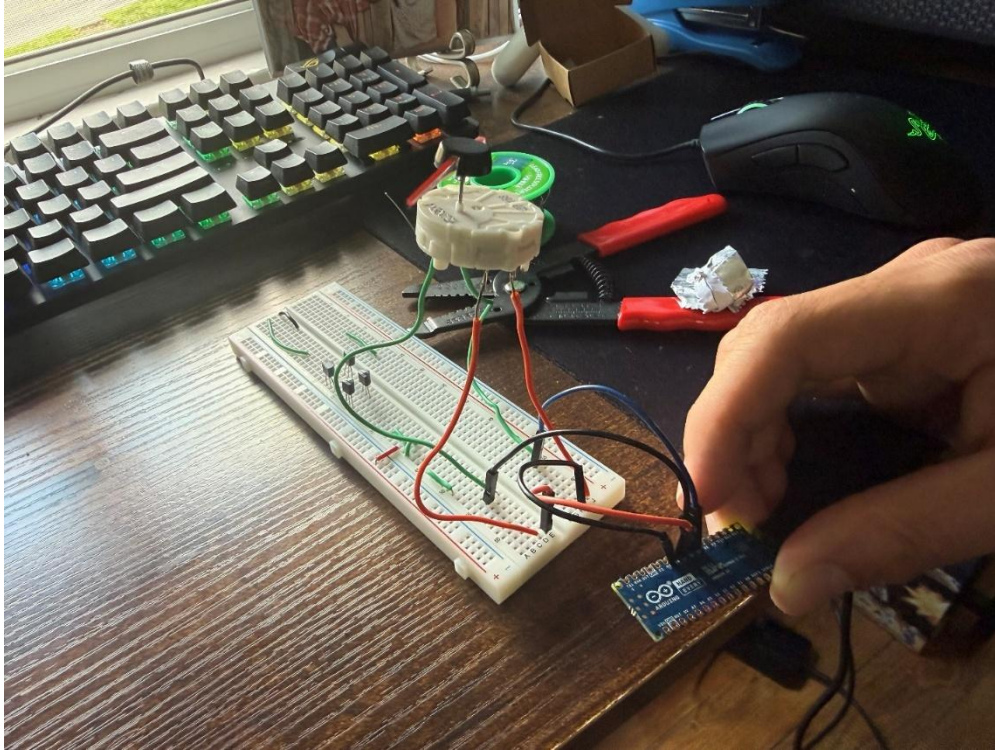


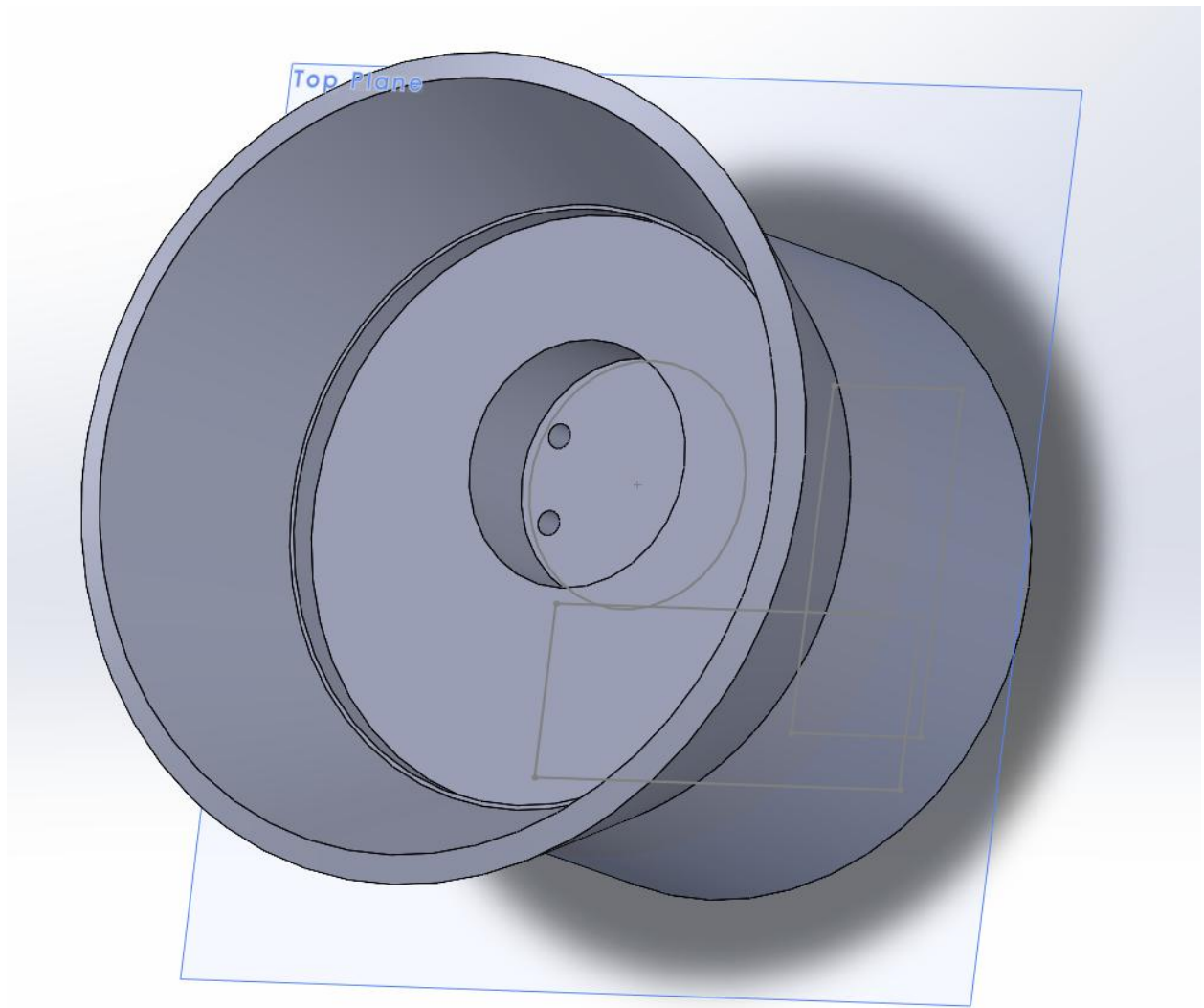
Photo of breadboard testing for the motor:



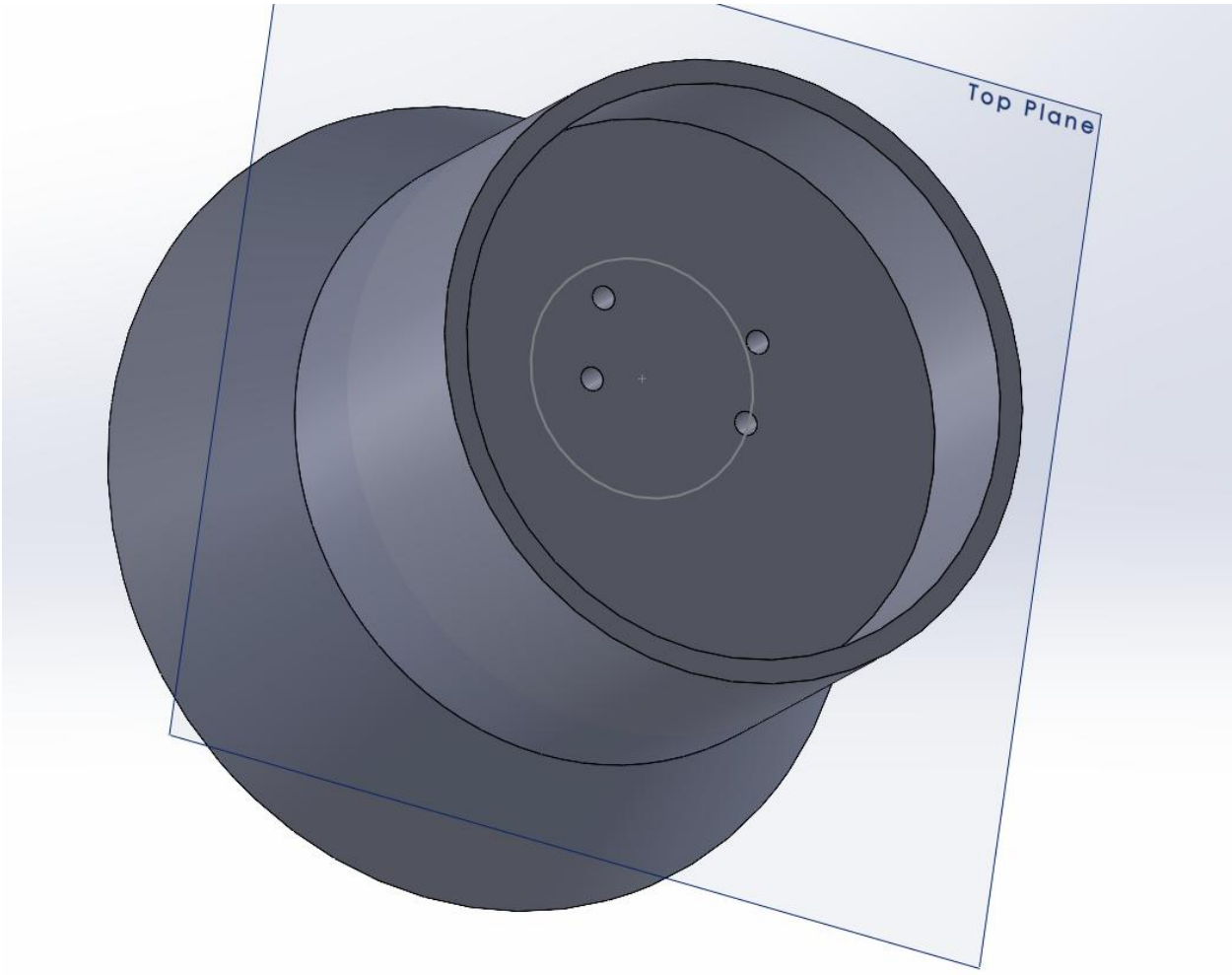
Housing for tachometer:

I designed a tachometer housing unit in solidworks for all my components. I'm planning on installing a light at the top of the unit to be able to see the dial even at night. I will cover the front and the back with plexiglass so that you can see the dial and the circuits in the back.

Front of tachometer housing:



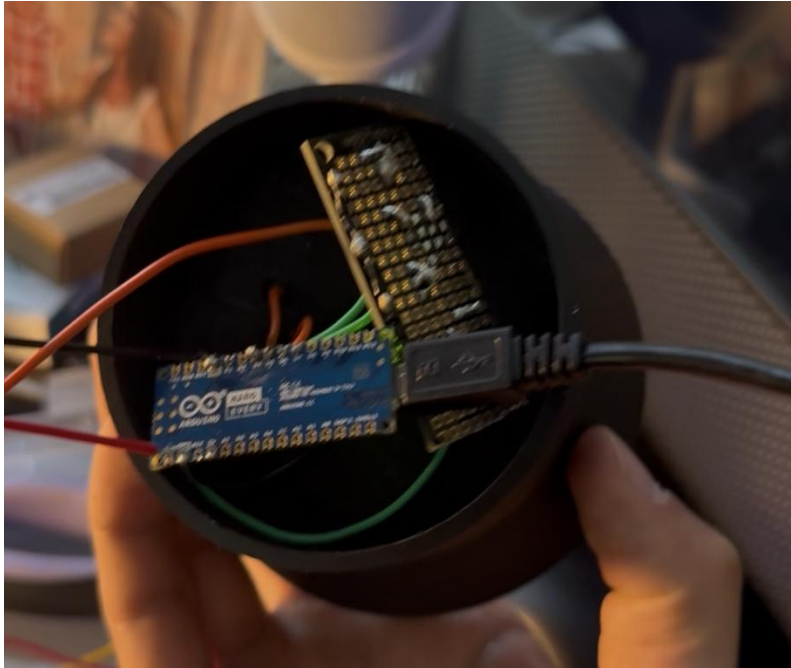
Back of tachometer housing:





This is a sticker I took off the internet with some slight modifications. I'm not planning on selling this and I don't know how to design a gauge that looks good, so for personal purposes this is fine. If I were to start selling these I would need to make my own sticker.

Back of Tachometer Housing:



Back with temporary cover:



Front of tachometer housing:



4. To do:

- a. Assemble Arduino, Voltage limiter circuit, and gauge dial
- b. Test and Verify